

Socket (TCP/IP)

6

통신 설정

PART 1

1. TCP/IP는 In/Out 주소에 직접 접근할 수 없습니다.
2. 직접 프로그래밍하여 상대 기기와 통신하여 데이터를 송수신합니다.
3. 로봇은 Client와 Server를 선택할 수 있으며, 최대 16개의 디바이스와 통신할 수 있습니다.

로봇을 Client or Server로 선택합니다.

프로그래밍 시 해당 ID로 포트 번호를 대신할 수 있습니다.

연결된 통신 상태를 나타냅니다.

ID	Controller	Remote device	Connection status
#2	Client:0:0	0:0	Disconnect
#3	Client:0:0	0:0	Disconnect
#4	Client:0:0	0:0	Disconnect
#5	Client:0:0	0:0	Disconnect
#6	Client:0:0	0:0	Disconnect
#7	Client:0:0	0:0	Disconnect
#8	Client:0:0	0:0	Disconnect
#9	Client:0:0	0:0	Disconnect
#10	Client:0:0	0:0	Disconnect
#11	Client:0:0	0:0	Disconnect
#12	Client:0:0	0:0	Disconnect
#13	Client:0:0	0:0	Disconnect
#14	Client:0:0	0:0	Disconnect
#15	Client:0:0	0:0	Disconnect
#16	Client:0:0	0:0	Disconnect
#17	Client:0:0	0:0	Disconnect

Port 2000 Open server Close server Disconnect Refresh list

1. Client 설정 시 로봇은 Client의 역할을 수행하며 포트가 열려 있는 Server에 접속할 수 있습니다.

Controller as

Controller as: ☒ Client ☐ Server

No.	ID	Controller	Remote device	Connection status
0	#2	Client:192.168.2.245:1234	192.168.2.240:1470	Connect
1	#3	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
2	#4	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
3	#5	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
4	#6	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
5	#7	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
6	#8	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
7	#9	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
8	#10	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
9	#11	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
10	#12	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
11	#13	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
12	#14	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
13	#15	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
14	#16	Client:0.0.0.0:0	---,---,---,---:0	Disconnect
15	#17	Client:0.0.0.0:0	---,---,---,---:0	Disconnect

Server IP
192.168.2.240

Server port
1470

Client port
1234

New connection

Disconnect Refresh list

접속할 Server의 IP를 입력합니다.

접속할 Server의 포트 번호를 입력합니다.

Client의 포트 번호를 설정합니다.

연결을 시도합니다.

1. Client 설정 시 로봇은 Server의 역할을 수행하며 포트 번호를 지정하고 포트를 열면 Client가 Server에 접속할 수 있습니다.

The screenshot shows a web-based configuration interface for a Socket Server. At the top, there is a section titled "Controller as" with two radio buttons: "Client" and "Server". The "Server" button is selected and highlighted with a red box. Below this is a table with four columns: "ID", "Controller", "Remote device", and "Connection status". The first row (#2) is highlighted with a red box and shows "Server:192.168.2.245:2000" for the Controller and "192.168.2.240:51290" for the Remote device, with a "Connect" status. The other rows show "Server:0.0.0.0:2000" and "Disconnect" status. At the bottom, there is a "Port" input field with "2000" entered, and a red box around it with the annotation "Server의 포트를 지정합니다.". Next to it is a red box around the "Open server" button with the annotation "지정한 포트 번호로 포트를 엽니다.". To the right, a red dashed line points from the first row of the table to a text box that says "Client와 통신이 이루어지면 해당 목록에 Client와 Server의 정보가 표시됩니다.".

Controller as

Controller as: ☐ Client ☒ Server

ID	Controller	Remote device	Connection status
#2	Server:192.168.2.245:2000	192.168.2.240:51290	Connect
#3	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#4	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#5	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#6	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#7	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#8	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#9	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#10	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#11	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#12	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#13	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#14	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#15	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#16	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect
#17	Server:0.0.0.0:2000	---,---,---,---:0	Disconnect

Server의 포트를 지정합니다.

지정한 포트 번호로 포트를 엽니다.

Client와 통신이 이루어지면 해당 목록에 Client와 Server의 정보가 표시됩니다.

Port 2000 Open server Close server Disconnect Refresh list

Socket 명령어

PART 2

1. Open Socket

로봇이 Client로 선택 시 사용됩니다. Server의 포트를 열 때 사용합니다.

Format 1: **Open Socket** ("IP", ServerPort, ClientPort, BufferType);

Format 2: **Open Socket** ("IP", ServerPort, ClientPort, BufferType, B[x]);

1) IP : Server IP 입력

2) ServerPort: Server Port 입력 (1 ~ 65535)

3) ClientPort: Client Port 입력 (2~5 or 1024 ~ 65535)

4) BufferType: Circle or Single 입력

- Circle : 위 명령어가 사용되면 피어로 부터 데이터를 수신합니다. 버퍼가 가득 차면 알람이 표시되며 새로운 데이터를 수신하지 않습니다.

Get Port 또는 GetAString 명령어를 사용해 데이터 취득 시 데이터가 수신되지 않으면 버퍼의 데이터가 누적됩니다.

- Single : Get Port 또는 GetAString 명령어를 사용해 데이터 취득 시 가장 최근에 수신한 데이터를 수신합니다.

- B[x] : B/LB 변수를 사용하며, 데이터 수신에 성공하면 1, 실패 시 0이 저장됩니다.

- 설명 :

1. Open Socket이 시작되면 포트는 항상 열려 있습니다. (닫을 시 Close Socket 명령어 사용하거나 수동으로 다른 프로그램을 열었을 때 자동으로 닫힙니다)

2. 로봇이 Client로 사용될 때 포트 범위는 1024~65535이며, 1111, 2222, 3333, 4444, 5555, 6666, 7777, 8888, 9999, 1217, 8080, 11740, 4003 포트 번호는 특별한 의미가 있는 포트 번호이기 때문에 해당 포트 번호를 사용할 때 주의해야 합니다.

```
LB[0] = 0;
While LB[0] <> 1 #통신 비정상일 경우 무한 반복
    Open Socket("192.168.2.240",1470,1234,Single,LB[0]);
    Delay T[0.2];
EndWhile;
```

1. Close Socket

열려 있는 포트를 닫을 때 사용합니다.

Format 1: **Close Socket, ClientPort;**

1) ClientPort: Client Port 입력 (2~5 or 1024 ~ 65535)

- 설명 :

1. 로봇이 Client/Server 일 때 주변 장치에 대한 포트를 닫을 때 사용합니다.

```
LB[0] = 0;
While LB[0] <> 1 #통신 비정상일 경우 무한 반복
    Open Socket("192.168.2.240",1470,1234,Single,LB[0]);
    Delay T[0.2];
EndWhile;
Send Port[1234],"Start"; #"Start" 문자열 데이터 송신
L[0]: #비전은 "Start"을 수신한 뒤 위치 값 "12.22, 3.2, 0, 0.5" 송신
Get Port[1234],T[1],Goto L[0]; #값을 수신할 때 까지 반복
Str[0] = GetPortbuf(0,100,1234); #수신한 데이터를 Str[0] 변수에 저장
Close Socket,1234;
```


1. Send Port

상대기기 포트에 문자열을 송신합니다.

Format 1: **Send Port[Port], String;**

Format 2: **Send Port[Port], String, Type;**

Format 3: **Send Port[Port], IntArrayData, IntNum;** (특주 펌웨어 전용)

- 1) Port: Client Port 입력 (1~5 or 1024 ~ 65535)
- 2) String: 보낼 문자열 입력
- 3) Type: 보낼 문자열 데이터를 해당 데이터 타입 (String, Hex) 으로 변환하여 전송합니다.
- 4) IntArrayData: 데이터를 문자열이 아닌 정수형으로 수신합니다.
- 5) IntNum: 정수 버퍼 영역에서 0~IntNum 만큼의 영역을 송신합니다.

- 설명 :

1. TCP/IP 통신 사용 시 Client/Server 모두 사용 가능하나 Format의 Port는 항상 Client 포트번호를 입력합니다.
2. 로봇이 Server일 때 통신하는 Client가 1개이고, 포트 번호를 알 수 없다면 4444포트 번호를 사용할 수 있습니다.
3. 멀티 테스크에서 여러 테스크에서 동일한 포트를 사용하여 동시에 데이터를 송/수신할 수 없습니다.

```
#상대 기기에 "ABC"를 송신
Send Port[2000], "ABC";

#문자열 변수를 사용하여 송신
Str[0] = "ABC";
Send Port[2000], Str[0];

#문자열을 16진수로 송신
Send Port[2000], "23EF5", Hex;
```

```
# 버퍼 0~39byte 영역에 aiSocket[0]~[9]의 값을 차례대로 저장
SetPortBufInt(2, aiSocket[0], 10, 0);
# 버퍼 0~3byte 영역에 aiSocket[0]의 값을 저장
SetPortBufInt(2, aiSocket[0], 0);
# 버퍼 0~39byte에 저장된 값을 송신
Send Port[2], IntArrayData, 40;
```

1. SetPortbufInt

정수 배열 변수에서 정수형 송신 버퍼로 이동/저장합니다.

Format 1: `SetPortbufInt(Port, StartArrVar, IntNum, BufStartAddr);`

Format 2: `SetPortbufInt(Port, BufStartAddr);`

- 1) Port: Client 포트 번호 입력 (1~5 or 1024~65535)
- 2) StartArrVar: 1차원 정수 배열 변수의 시작 주소 (ArrVar[0]~ArrVar[256])
- 4) IntNum: 버퍼 메모리에 저장할 영역을 설정합니다. (1~256)
- 5) BufStartAddr: 정수 버퍼에 저장할 선두 번지를 지정합니다.

- 설명 :

- 1. INT 배열 변수에서 원하는 만큼의 정수 데이터를 4byte 씩 구분되어 정수 수신 버퍼에 저장합니다.
- 2. SetPortbufInt 사용 시 Send Port 명령어는 IntArrayData 형식을 사용해야 합니다.

```
# 버퍼 0~39byte 영역에 aiSocket[0]~[9]의 값을 차례대로 저장
SetPortBufInt(2, aiSocket[0], 10, 0);
# 버퍼 0~3byte 영역에 aiSocket[0]의 값을 저장
SetPortBufInt(2, aiSocket[0], 0);
# 버퍼 0~39byte에 저장된 값을 송신
Send Port[2],IntArrayData,40;
```

1. Get Port

해당 포트에서 문자열을 수신합니다.

Format 1 : `Get Port[Port], T[sec], Goto L[Line number];`

Format 2 : `Get Port[Port], IntArrayData, T[sec], Goto L[Line number];` (특주 펌웨어 전용)

- 1) Port: Client Port 입력 (1~5 or 1024 ~ 65535)
- 2) IntArrayData : 데이터를 문자열이 아닌 정수형으로 수신합니다.
- 3) T[sec]: 수신을 유지할 시간을 설정합니다. (0 ~ 65535, Unit : s)
- 4) Goto L: 설정 시간 안에 데이터가 수신되지 않았을 경우 Line 점프할 Line을 입력합니다.

- 설명 :

1. TCP/IP 통신 사용 시 Client/Server 모두 사용 가능하나 Port는 항상 Client 포트번호를 입력합니다.
2. 로봇이 Server일 때 Client가 1개이고, 포트 번호를 모를 경우 4444포트 번호를 사용할 수 있습니다.
3. 설정 시간 동안 데이터를 수신합니다. 수신 성공 시 수신 버퍼에 저장되고 다음 명령행이 실행됩니다.
수신 실패 시 Goto L이 실행되어 L으로 점프합니다.
4. Open Socket 명령의 BufferType이 Circle으로 설정되어 있다면 Get Port 명령은 마지막 명령 이후의 모든 데이터를 가져옵니다.
Single로 설정 시에 Get Port 명령은 가장 최근에 수신된 데이터를 가져옵니다. 가져온 버퍼 데이터는 획득 후 자동으로 지워집니다.

```
# Open Socket이 Circle로 사용될 경우
# 상대 기기가 "ABC"를 보낸 뒤 "EFG"를 추가로 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "ABCEFG"
```

```
# 상대 기기가 "ABC"를 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "ABC"
```

```
# 상대 기기가 "EFG"를 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "EFG"
```

```
# Open Socket이 Single로 사용될 경우
# 상대 기기가 "ABC"를 보낸 뒤 "EFG"를 추가로 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "EFG"
```

```
# 정수형으로 데이터 수신
Get Port[2],T[1],IntArrayData,Goto L[0];
# aiSocket[0]~[9]에 버퍼 0~39byte 영역 값을 차례대로 저장
R[0] = GetPortbufInt(2, aiSocket[0], 10, 0);
# aiSocket[0]에 버퍼 4~7byte 영역 값 저장
aiSocket[0] = GetPortbufInt(2, 4);
```

1. GetPortbuf

수신 버퍼에서 문자열을 문자열 변수로 이동/저장합니다.

Format: **StrVar = GetPortbuf(StartBit, Number, Port);**

- 1) StartBit: 가져올 버퍼의 시작 주소 (0~1023)
- 2) Number: 가져올 버퍼의 문자열 길이 (1~100)
- 3) Port: Client 포트 번호 입력 (1~5 or 1024~65535)

- 설명 :

1. 수신 버퍼의 StartBit에서 부터 Number 길이까지의 문자열은 가져와 지정한 문자열 변수에 저장합니다. 실패 시 0이 저장됩니다.
2. GetPortbuf 사용 전 Get Port 명령어가 우선적으로 실행되어야 합니다. 그렇지 않으면 가져온 문자열이 빈 문자열이기 때문에 오류가 발생합니다.

```
# Open Socket이 Circle로 사용될 경우
# 상대 기기가 "ABC"를 보낸 뒤 "EFG"를 추가로 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "ABCEFG"

# 상대 기기가 "ABC"를 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "ABC"

# 상대 기기가 "EFG"를 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "EFG"
```

```
# Open Socket이 Single로 사용될 경우
# 상대 기기가 "ABC"를 보낸 뒤 "EFG"를 추가로 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
# Str[0] == "EFG"
```

1. GetPortbufInt

정수형 수신 버퍼에서 정수 배열 변수로 이동/저장합니다.

Format 1: `ByteSize = GetPortbufInt(Port, StartArrVar, IntNum, BufStartAddr);`

Format 2: `ArrVar[n] = GetPortbufInt(Port, BufStartAddr);`

- 1) ByteSize: 가져온 데이터의 Byte 개수를 저장합니다. (Ex Int 3개 수신 시 12 저장)
- 2) Port: Client 포트 번호 입력 (1~5 or 1024~65535)
- 3) StartArrVar: 저장할 1차원 정수 배열 변수의 시작 주소 (ArrVar[0]~ArrVar[256])
- 4) IntNum: 1차원 정수 배열 변수에 저장할 영역을 설정합니다. (1~256)
- 5) BufStartAddr: 정수 버퍼에서 가져올 선두 번지를 지정합니다. (0~1020)
- 6) ArrVar[n]: 4byte를 수신할 1차원 INT 배열 변수를 지정합니다.

- 설명 :

1. 정수 수신 버퍼에서 원하는 만큼의 정수 데이터를 4byte 씩 구분되어 INT 배열 변수에 저장합니다.
2. GetPortbufInt 사용 전 Get Port 명령어가 우선적으로 실행되어야 합니다. Get Port 명령어는 IntArrayData 형식을 사용해야 합니다.

```
# 정수형으로 데이터 수신
Get Port[2],T[1],IntArrayData,Goto L[0];
# aiSocket[0]~[9]에 버퍼 0~39byte 영역 값을 차례대로 저장
R[0] = GetPortbufInt(2, aiSocket[0], 10, 0);
# aiSocket[0]에 버퍼 4~7byte 영역 값 저장
aiSocket[0] = GetPortbufInt(2, 4);
```


1. StrGetData

로봇에서 수신하는 데이터를 구분자를 통해 값을 별도의 변수에 저장합니다.

Format: **Num = StrGetData(StrSource, Separator, Data);**

- 1) Num: 구분된 문자열 개수를 저장합니다.
- 2) StrSource: 구분할 문자열을 지정합니다. (Str 변수나 Port에서 수신한 데이터만 사용할 수 있습니다.)
- 3) Separator: 구분자를 지정합니다.
- 4) Data: 숫자 변수를 사용하여 구분자를 통해 구분된 데이터를 지정한 선두 번지에서 차례대로 다음 변수에 저장합니다.

- 설명 :

1. 숫자가 아닌 데이터가 수신되면 '0'값을 저장합니다.
2. 구분자는 문자열로 지정합니다.
3. Data에 사용할 수 있는 변수는 B, R, D, LB, LR, LD, 사용자 변수(배열)으로 사용할 수 있습니다.

```
# 상대 기기가 "123,456,789" 보낸 경우
Get Port[1025],T[1],Goto L[0];
Str[0] = GetPortbuf(0,100,1025);
B[0] = StrGetData(Str[0],",",R[0])
# B[0] = 3
# R[0] = 123
# R[1] = 456
# R[2] = 789
```

1. GetAString

로봇에서 수신하는 데이터를 구분자를 통해 값을 구분해서 수신하거나 전체로 수신하여 Str[] 변수에 저장합니다.

Format: **GetAString Port[n], T[n], Str[n], "Sep", Goto L[n];**

- 1) Port: 클라이언트 포트 번호를 입력합니다.
- 2) T: 데이터를 수신할 시간을 설정합니다.
- 3) Separator: 구분자를 지정합니다.
- 4) Goto L: 설정된 시간에 데이터를 수신하지 못하면 설정한 L 라벨로 이동합니다.

- 설명 :

1. Open Socket이 Single이면 GetAString 명령은 Get Port + Getportbuf와 동일한 기능입니다. (구분자 무시됨)
2. Open Socket이 Circle로 설정되면 수신한 데이터에 설정한 구분자가 포함되는지 확인하고 구분자가 있을 경우 구분자와 구분자 앞 문자열을 구분하여 수신합니다. 다시 해당 명령어를 실행하면 다음 문자열을 재검색합니다.

```
Open Socket("192.168.1.62",1234,2000,Circle,B[0]);
B[1] = 0;
L[0]:
"123,456,78," 송신 시 (Circle)
While B[1] <= 3
    GetAString Port[2000],T[1],Str[0],"",Goto L[0];
    Print Str[0];
    R[B[1]] = StrToR(Str[0]);
    Incr B[1];
EndWhile;
R[0] = 123 R[1] = 456 R[2] = 78
Close Socket,2000;
```

Output

```
[Print ]: 18:21:20 123,
[Print ]: 18:21:35 456,
[Print ]: 18:21:43 78,
```

```
Open Socket("192.168.1.62",1234,2000,Single,B[0]);
B[1] = 0;
L[0]:
"123,456,78," 송신 시 (Single)
    GetAString Port[2000],T[1],Str[0],"",Goto L[0];
    Print Str[0];
    Str[0] = 123,456,78,
Close Socket,2000;
```

1. GetPortState

지정한 포트의 통신 상태를 가져옵니다.

Format: **Var = GetPortState(PortNo);**

1) PortNo: Client 포트 번호 입력 (1~5 or 1024~65535)

- 설명 :

1. 통신하는 포트의 통신 상태를 확인할 수 있습니다. 반환 값이 0일 경우 통신 불량, 1일 경우 통신 정상

```
B[0] = 0;
L[0]:
B[0] = GetPortState(2000);
If B[0] == 0
    Print "통신 실패";
    Goto L[0];
Else
    Print "통신 성공";
EndIf;
```

프로그램 예제 (Socket - TCP/IP)

PART 4

1. Robot IP : 192.168.2.245 / Port : 1234 (Client)
2. Server IP : 192.168.2.240 / Port : 1470 (Server)
3. 로봇과 비전이 통신하여 로봇의 offset 위치를 수신하는 예제

```
#비전과 TCP/IP 소켓 통신 (로봇 위치 offset)
#로봇이 Client일 경우
LB[0] = 0;
While LB[0] <> 1 #B[0]이 1(통신 비정상일 경우 무한 반복)
    Open Socket("192.168.2.240",1470,1234,Single,LB[0]);
    Delay T[0.2];
EndWhile;

Send Port[1234],"Start"; #"Start" 문자열 데이터 송신
L[0]: #비전은 "Start"을 수신한 뒤 위치 값 "12.22, 3.2, 0, 0.5" 송신
Get Port[1234],T[1],Goto L[0]; #값을 수신할 때 까지 반복
Str[0] = GetPortbuf(0,100,1234); #수신한 데이터를 Str[0] 변수에 저장
B[0] = StrGetData(Str[0],",",D[0]); #", "를 구분하여 D[0]~D[3]에 값 분류
#B[0] = 4 (4개의 데이터로 분류)
#D[0] = 12.22 (X Offset 위치)
#D[1] = 3.2 (Y Offset 위치)
#D[2] = 0 (Z Offset 위치)
#D[3] = 0.5 (A Offset 위치)
PR[0] = (D[0],D[1],D[2],D[3],0,0); #PR[0] 변수에 각 축의 위치값 저장
Movj Offset(P[0],PR[0]),V[100],Z[0],Tool[0],Wobj[0]; #Offset 위치로 이동
```


1. Client IP : 192.168.2.240 / Port : 2000 (Client)
2. Robot IP : 192.168.2.245 / Port : 3000 (Server)
3. 로봇과 비전이 통신하여 로봇의 offset 위치를 수신하는 예제

```
#비전과 TCP/IP 소켓 통신 (로봇 위치 offset)
#로봇이 Server일 경우
While GetPortState(2000) <> 1 #통신 비정상일 경우 무한 반복
    Print "Comm Failed";
    Delay T[0.5];
EndWhile;

Send Port[2000],"Start"; #"Start" 문자열 데이터 송신
L[0]: #비전은 "Start"을 수신한 뒤 위치 값 "12.22, 3.2, 0, 0.5" 송신
Get Port[2000],T[1],Goto L[0]; #값을 수신할 때 까지 반복
Str[0] = GetPortbuf(0,100,2000); #수신한 데이터를 Str[0] 변수에 저장
B[0] = StrGetData(Str[0],",",D[0]); #","를 구분하여 D[0]~D[3]에 값 분류
#B[0] = 4 (4개의 데이터로 분류)
#D[0] = 12.22 (X Offset 위치)
#D[1] = 3.2 (Y Offset 위치)
#D[2] = 0 (Z Offset 위치)
#D[3] = 0.5 (A Offset 위치)
PR[0] = (D[0],D[1],D[2],D[3],0,0); #PR[0] 변수에 각 축의 위치값 저장
Movj Offset(P[0],PR[0]),V[100],Z[0],Tool[0],Wobj[0]; #Offset 위치로 이동
```

INOVANCE

Forward, Always Progressing